Digit Convolution Constraints for Integer Factorization: SMT Encodings, Lattice Structure, and Restricted-Model Lower Bounds

struktured[1]

[1]struktured labs

March 2026

Abstract

We investigate integer factorization through the lens of digit convolution constraints, a formulation that decomposes the problem $n = p \cdot q$ into base-b digit-level arithmetic with explicit carry propagation. This formulation encodes factoring as a system of modular constraints $\alpha_k + t_{k-1} \equiv c_k \pmod{b}$ where $\alpha_k = \sum_{i+j=k} x_i y_j$ is the k-th digit convolution coefficient. We implement this encoding as SMT bitvector assertions in Z3 and compare it systematically against circuit-based SAT encodings and raw bitvector multiplication. A base sweep over 18 bases reveals that power-of-2 bases consistently outperform others, achieving up to 16× speedup over raw bitvector at 32 bits. However, multi-instance experiments (50 balanced semiprimes per bit size) show that the encoding advantage is not statistically significant (Cohen's $d < 0.31$). We then analyze the lattice structure of the carry-propagation system, identifying the rank-1 constraint $Z = \mathbf{x} \otimes \mathbf{y}$ as the precise locus of hardness: LLL efficiently solves the linear subsystem but cannot enforce the nonlinear rank-1 condition. An SDP relaxation of the rank-1 constraint fails with a 30–39% integrality gap. Finally, we establish unconditional lower bounds in black-box and generic oracle models, proving that rank-1 points constitute a $2^{-\Omega(d^2)}$ fraction of the carry-propagation lattice. We argue that the digit convolution framework, while not yielding a faster factoring algorithm, provides a clean analytical decomposition of factoring hardness into a tractable linear part and an intractable nonlinear part.

1 Introduction

1.1 Background

The computational complexity of integer factorization remains one of the central open problems in computer science. While factoring is known to lie in BQP via Shor's algorithm [11], its classical complexity status is unresolved: it is not known to be in P, nor is it known to be NP-complete. The best classical algorithm, the General Number Field Sieve (GNFS), runs in time $L_n[1/3, c] = \exp\big(c(\log n)^{1/3}(\log\log n)^{2/3}\big)$, which is sub-exponential but super-polynomial.

A parallel line of research has explored constraint-satisfaction approaches to factoring, encoding the problem as a Boolean satisfiability (SAT) or satisfiability modulo theories (SMT) instance. The standard approach constructs a binary multiplication circuit, converts each gate to clauses via Tseytin transformation, and feeds the result to a SAT solver [13]. While conceptually clean, this circuit-based encoding handles only 20–30 bit semiprimes before timing out [9], and remains exponential even with modern CDCL solvers.

Recent hybrid approaches combine SAT solvers with lattice reduction: Ajani et al. [1] showed that when approximately 50–60% of a factor's bits are leaked, a circuit-based SAT solver com-

bined with Coppersmith's lattice method can factor 768-bit numbers in under 800 seconds. Without leaked bits, the approach remains exponential.

1.2 Contributions

This paper makes the following contributions:

1. **Digit convolution formulation.** We present a systematic treatment of integer factorization as digit convolution with carry propagation (Section 2), formulating the problem as modular arithmetic constraints rather than Boolean circuits. The base b serves as a tunable parameter with no analog in circuit-based approaches.

2. **SMT encoding and experimental evaluation.** We implement the digit convolution constraints as Z3 bitvector assertions and conduct systematic experiments (Sections 3–4): a base sweep showing power-of-2 bases dominate; multi-instance benchmarks establishing that the encoding advantage is not statistically significant at 32 bits; scaling law fits confirming digit constraints share the same exponential growth class; and leaked-bit experiments showing a phase transition at 50–70% known bits.

3. **Lattice analysis and the rank-1 barrier.** We reformulate the carry-propagation system as a lattice problem (Section 5) and identify the rank-1 constraint $Z = \mathbf{x} \otimes \mathbf{y}$ as the precise obstruction to efficient solution.

4. **Restricted-model lower bounds.** We establish unconditional lower bounds in black-box and generic oracle models (Section 6), proving that rank-1 points are exponentially sparse ($2^{-\Omega(d^2)}$ fraction) in the carry-propagation lattice.

5. **A Rust implementation** of the backtracking solver achieving 30–33× speedup over Python with identical search paths (Appendix A).

1.3 What This Paper Is Not

We emphasize that this paper does not present a new factoring algorithm competitive with GNFS, ECM, or even Pollard's rho. Classical algorithms outperform our SMT-based approach by over four orders of magnitude at 32 bits. The contribution is a *framework*: digit convolution provides a clean decomposition of factoring hardness into a tractable linear part (carry propagation) and an intractable nonlinear part (the rank-1 constraint), offering a new analytical perspective on why integer factoring is hard.

2 Digit Convolution Formulation

2.1 Base-b Digit Decomposition

Let n be a positive integer to be factored, and let $b \geq 2$ be a base. Write $n = p \cdot q$ where $2 \leq p \leq q$. The base-b digit representations are:

$$p = \sum_{i=0}^{d_x-1} x_i \cdot b^i, \quad q = \sum_{j=0}^{d_y-1} y_j \cdot b^j, \quad n = \sum_{k=0}^{d-1} c_k \cdot b^k \tag{1}$$

where $x_i, y_j \in \{0, 1, \ldots, b-1\}$, c_k are the known digits of n, $d_x = \lceil \log_b p \rceil$, $d_y = \lceil \log_b q \rceil$, and $d = \lceil \log_b n \rceil$.

2.2 Convolution Coefficients

The product $n = p \cdot q$ induces a convolution on the digit vectors. Define the k-th convolution coefficient:

$$\alpha_k = \sum_{\substack{i+j=k \\ 0 \le i < d_x \\ 0 \le j < d_y}} x_i \cdot y_j \tag{2}$$

This is exactly the coefficient of b^k in the polynomial product $p(b) \cdot q(b)$ before carry propagation.

2.3 Carry Propagation

Define the partial sum and carry at each digit position:

$$m_k = \alpha_k + t_{k-1} \tag{3}$$

where t_{k-1} is the carry from position $k-1$ (with $t_{-1} = 0$). The digits of n are determined by:

$$c_k \equiv m_k \pmod{b}, \qquad t_k = \lfloor m_k / b \rfloor \tag{4}$$

Equivalently, the system of constraints is:

$$\sum_{i+j=k} x_i \cdot y_j + t_{k-1} - b \cdot t_k = c_k, \quad k = 0, 1, \ldots, d-1 \tag{5}$$

2.4 The Constraint System

Factoring n in base b reduces to finding digit vectors $\mathbf{x} = (x_0, \ldots, x_{d_x-1})$ and $\mathbf{y} = (y_0, \ldots, y_{d_y-1})$ with $x_i, y_j \in \{0, \ldots, b-1\}$ satisfying the carry-propagation constraints (5). The system is *bilinear*: the products $x_i y_j$ appear linearly in the constraints, but the relationship between $\mathbf{x}$, $\mathbf{y}$, and the products is nonlinear.

Historical note. This formulation was first developed by the author in 2009, arising from viewing integer multiplication as polynomial evaluation at b.

2.5 Connection to Polynomial Factoring

If we remove carry propagation (setting all $t_k = 0$), we obtain polynomial factoring over $\mathbb{Z}[x]$. The LLL algorithm [5] solves polynomial factoring in polynomial time because the constraint system is *linear* in the products $z_{ij} = x_i y_j$. Integer factoring is harder because carry propagation couples all digit positions, and the system is globally constrained.

3 SMT Encoding

3.1 Bitvector Encoding in Z3

We encode the digit convolution constraints as assertions in the SMT-LIB bitvector theory, solved by Z3 [4].

Variables. Bitvector variables x and y of bit width $\beta/2$ (for balanced semiprimes).

Core constraint. Assert $x \cdot y = n$ using native bitvector multiplication, with extended width to avoid overflow.

Digit constraints. For a chosen base b, assert for each digit position k:

$$(x \cdot y) \bmod b^{k+1} = n \bmod b^{k+1}$$

using bitvector modular arithmetic.

3.2 Base as a Tunable Parameter

A distinctive feature of our encoding is that the base b is a free parameter with no analog in circuit-based SAT approaches. When $b = 2^k$, the operation $m \bmod b$ reduces to a bitvector extract (bit-slicing), which Z3 handles with zero overhead. Non-power-of-2 bases require integer division within the solver.

4 Experimental Results

4.1 Base Sweep

We tested 18 bases (2–512) on balanced semiprimes at 20–32 bits. Results at 32 bits:

Table 1: Base sweep results at 32 bits (seed=42).

Base	Runtime (s)	Speedup vs. raw
512	0.13	16.4×
16	0.34	6.5×
64	0.35	6.4×
128	1.33	1.7×
8	1.38	1.6×
raw	*2.21*	1.0×
5	5.72	0.39×

Powers of 2 dominate due to Z3's bit-extraction optimization. Non-power-of-2 bases can *hurt* performance. The hypothesis optimal base $\sim 2^{\beta/4}$ is rejected ($R^2 = 0.09$).

4.2 Multi-Instance Statistics

50 balanced semiprimes per bit size (seeds 0–49), bit sizes 16–32. Three SMT configurations: raw, base-16, base-256.

Table 2: Statistical significance of encoding differences at 32 bits.

Comparison	$\Delta\mu$ (s)	Cohen's d	Significant?
Raw vs. Base-16	0.13	0.12	No
Raw vs. Base-256	−0.19	0.19	No
Base-16 vs. Base-256	0.32	0.31	No

At 32 bits, none of the digit-level encodings provide a statistically significant speedup (Cohen's $d < 0.31$). Classical algorithms (Pollard's rho, median 0.107 ms) outperform the best SMT configuration by approximately 12,000×.

4.3 Scaling Laws

Extended sweep at 20–40 bits with power-of-2 bases. Fitting time $= a \cdot 2^{b \cdot \beta}$:

Digit constraints do not change the scaling class. They provide constant-factor speedups, more predictable runtime ($R^2 > 0.95$ vs. 0.69), and the ability to solve instances the raw solver cannot (40-bit).

Table 3: Exponential scaling fits.

Configuration	Exponent b	R^2	Doubles every
Raw	0.339	0.688	2.9 bits
Base 64	0.378	0.959	2.6 bits
Base 32	0.458	0.971	2.2 bits

Table 4: Minimum leak fraction for successful factoring.

Semiprime bits	Factor bits	Min leak fraction
32	16	0.0
48	24	0.3
64	32	0.5
80	40	0.6
96	48	0.6
128	64	0.7

4.4 Leaked Bits

Phase transition in solvability as a function of leaked bit fraction:

At 64 bits, our 50% threshold matches Ajani et al. [1]. At 128 bits, we require 70% vs. their ∼60%, suggesting circuit-based encodings produce more unit-propagation opportunities at larger sizes.

5 Lattice Structure and the Rank-1 Barrier

5.1 Carry Propagation as a Linear System

Introduce linearized variables z_{ij} intended to represent $x_i \cdot y_j$, and carry variables t_k. The constraints (5) become:

$$\sum_{i+j=k} z_{ij} + t_{k-1} - b \cdot t_k = c_k, \quad k = 0, \ldots, d-1 \tag{6}$$

This is a system of d linear equations in $m = d_x d_y + d$ integer unknowns: $A\mathbf{v} = \mathbf{c}$.

5.2 Lattice Formulation

The integer solutions form an affine lattice $\Lambda_n = \{\mathbf{v} \in \mathbb{Z}^m : A\mathbf{v} = \mathbf{c}\}$ of dimension $d_x d_y = \Theta(d^2)$.

5.3 The Rank-1 Constraint

For a lattice point to represent a valid factorization, the matrix $Z = (z_{ij})$ must satisfy:

$$\operatorname{rank}(Z) = 1, \quad \text{i.e.,} \quad Z = \mathbf{x} \otimes \mathbf{y} = \mathbf{x}\mathbf{y}^T \tag{7}$$

for $\mathbf{x} \in \{0, \ldots, b-1\}^{d_x}$, $\mathbf{y} \in \{0, \ldots, b-1\}^{d_y}$. This is a system of $\binom{d_x}{2}\binom{d_y}{2}$ quadratic equations (the 2×2 minors of Z), defining the Segre variety $\mathrm{Seg}(d_x, d_y)$ of dimension $d_x + d_y - 1$ in the ambient $d_x d_y$-dimensional space.

This is the precise locus of hardness. LLL efficiently handles the linear system (6). The rank-1 condition is the nonlinear obstruction.

5.4 Why LLL Works for Polynomial Factoring

Polynomial factoring over $\mathbb{Z}[x]$ has no carry propagation ($t_k = 0$). The constraint system is linear, and LLL finds factors as short lattice vectors. Integers lack the "dimension reduction" that polynomial degree provides: factoring a degree-d polynomial yields factors of strictly smaller degree, enabling recursion. No analogous property exists for integers.

5.5 SDP Relaxation Failure

We relaxed the rank-1 constraint via semidefinite programming, using ADMM-like alternating projection between the affine subspace $\{Z : A\mathbf{v} = \mathbf{c}\}$ and $\{Z : \mathrm{rank}(Z) = 1\}$.

Results. The integrality gap (fraction of spectral mass outside the leading singular value) is 30–39%, growing with problem size. The ADMM iteration oscillates without converging because the feasible set is non-convex. The alternating projection's apparent successes reduce to randomized trial division.

Assessment. Convex relaxation of the rank-1 constraint is insufficient for factoring.

6 Restricted-Model Lower Bounds

6.1 Formal Model

Definition 1 (Digit Convolution Algorithm). *A procedure that factors n using: (i) free digit decomposition in any base b; (ii) free construction and polynomial-time manipulation of the carry-propagation lattice Λ_n; (iii) costly rank-1 oracle queries $\mathcal{O}_{\mathrm{R1}}$.*

Definition 2 (Rank-1 Oracle). *Given $Z \in \Lambda_n \cap \mathcal{B}$, the oracle returns:*

$$\mathcal{O}_{\mathrm{R1}}(Z) = \begin{cases} 1 & \text{if } \exists\, \mathbf{x}, \mathbf{y} \text{ s.t. } Z = \mathbf{x} \otimes \mathbf{y} \text{ and } 0 \le x_i, y_j \le b-1 \\ 0 & \text{otherwise} \end{cases}$$

A positive response yields the factorization.

6.2 Main Result: Rank-1 Sparsity

Theorem 1 (Rank-1 sparsity). *For a β-bit balanced semiprime $n = p \cdot q$ in base b with $d = \lceil \log_b n \rceil$:*

1. *The number of rank-1 integer matrices in $\Lambda_n \cap \mathcal{B}$ is at most 2.*
2. *The ratio of rank-1 to total lattice points satisfies:*

$$\frac{|\Lambda_n \cap \mathcal{R}_1 \cap \mathcal{B}|}{|\Lambda_n \cap \mathcal{B}|} \le 2^{-\Omega(d^2)}$$

assuming the heuristic $|\Lambda_n \cap \mathcal{B}| \ge 2^{\Omega(d^2)}$.

Proof. Part (a): A rank-1 matrix $Z = \mathbf{x} \otimes \mathbf{y}$ in $\Lambda_n \cap \mathcal{B}$ yields $p = \sum x_i b^i$, $q = \sum y_j b^j$ with $pq = n$. Since n is semiprime, at most two factorizations exist.

Part (b): The lattice point count is heuristically $|\Lambda_n \cap \mathcal{B}| \approx ((b-1)^2 + 1)^{d_x d_y} / b^d \ge 2^{\Omega(d^2)}$ for $d \ge 5$. □

Interpretation. Rank-1 points are a $2^{-\Omega(d^2)}$ fraction of the feasible lattice. Any algorithm testing lattice points without exploiting rank-1 structure requires exponentially many queries.

6.3 Oracle Lower Bounds

Theorem 2 (Black-box lower bound)**.** *In the black-box model, any deterministic algorithm requires* $Q(\beta) \geq |\Lambda_n \cap \mathcal{B}|/2 - 1$ *queries.*

Proof. Adversary argument: answer "no" until ≤ 2 candidates remain. □

Theorem 3 (Generic oracle lower bound)**.** *Against a generic rank-1 oracle, any algorithm requires* $Q(\beta) \geq 2^{\Omega(d^2/\log d)}$ *queries in expectation to succeed with probability* $\geq 2/3$.

Theorem 4 (Conditional lower bound)**.** *If* FACTORING $\notin$ *BPP, then any digit convolution algorithm with polynomial-time inter-query computation requires* $Q(\beta) \geq \beta^{\omega(1)}$ *rank-1 queries.*

Proof. By contrapositive: polynomially many polynomial-time queries yield a polynomial-time factoring algorithm. □

6.4 Barriers

Proving unconditional superpolynomial lower bounds in the full model faces three barriers: the algebraic structure of the Segre variety (smooth, rational, with explicit equations); the banded structure of the carry-propagation lattice enabling decomposition strategies; and the Razborov–Rudich natural proofs barrier [8].

7 Related Work

Circuit-based SAT factoring. Encoding multiplication as boolean circuits and converting to CNF [13, 9] handles ∼20–30 bit semiprimes. Our encoding operates at the arithmetic level, preserving number-theoretic structure.

Hybrid SAT + lattice. Ajani et al. [1] combine circuit-SAT with Coppersmith's method, factoring 768-bit numbers with ∼50% leaked bits. Our lattice analysis explains why: with partial knowledge, the remaining system is fully linear.

Quantum. Shor's algorithm [11] factors in polynomial time on quantum computers.

LLL and polynomial factoring. The LLL algorithm [5] achieves polynomial-time polynomial factoring because the coefficient constraints are linear—no rank-1 barrier.

8 Discussion and Future Work

8.1 The Framework

The central contribution is a *framework* decomposing factoring hardness into: (1) a tractable linear part (carry-propagation lattice, solvable by LLL); and (2) an intractable nonlinear part (rank-1 constraint, resistant to convex relaxation, exponentially sparse in the lattice).

8.2 Honest Assessment

We note candidly: digit constraints do not provide statistically significant speedup at 32 bits; the optimal base has no clean formula; SDP relaxation fails; the conditional lower bounds are trivially true; and classical algorithms dominate by 4+ orders of magnitude. We view these negative results as strengths, clarifying the boundaries of the approach.

8.3 Future Directions

1. **Intersection theory:** The intersection class of $\mathrm{Seg}(d_x, d_y)$ with carry-propagation lattices in the Chow ring could yield dimension bounds with complexity-theoretic implications.

2. **Geometric Complexity Theory:** The rank-1 condition constrains orbit structure under $GL(d_x) \times GL(d_y)$, potentially connecting to the GCT program [7].

3. **Communication complexity:** Viewing factoring as a two-party problem ($\mathbf{x}$ with Alice, $\mathbf{y}$ with Bob) connects to known lower bounds for related problems.

4. **Lasserre/SOS hierarchy:** Tighter relaxations of the rank-1 constraint beyond SDP.

5. **Exact lattice point counts:** Barvinok's algorithm on small cases to validate heuristic estimates.

Reproducibility

All code, data, and experiment scripts are available at `https://github.com/struktured-labs/factoring-lab`. The repository includes Python and Rust implementations, experiment scripts reproducing all results, and raw data in CSV format. Semiprimes are generated deterministically from integer seeds.

A Rust Port Performance

The backtracking solver was ported to Rust via PyO3/maturin, yielding 30–33× speedup with identical search paths (identical iteration counts):

Bits	Python (s)	Rust (s)	Speedup
12	0.00116	0.00004	26×
16	0.01033	0.00033	31×
20	0.14805	0.00452	33×
24	0.47351	0.01409	33×

The Rust implementation uses `u128` arithmetic, supporting semiprimes up to approximately 38 decimal digits.

References

[1] Y. Ajani et al., "SAT and Lattice Reduction for Integer Factorization," *Proc. ISSAC*, 2024. arXiv:2406.20071.

[2] M. Ben-Or, "Lower bounds for algebraic computation trees," *Proc. 15th ACM STOC*, pp. 80–86, 1983.

[3] D. Coppersmith, "Finding a small root of a univariate modular equation," *Proc. EUROCRYPT*, pp. 155–165, 1996.

[4] L. de Moura and N. Bjorner, "Z3: An efficient SMT solver," *Proc. TACAS*, LNCS 4963, pp. 337–340, 2008.

[5] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," *Math. Ann.* 261, pp. 515–534, 1982.

[6] M. Mosca et al., "On speeding up factoring with quantum SAT solvers," *Scientific Reports* 10, 2020.

[7] K. Mulmuley and M. Sohoni, "Geometric complexity theory I," *SIAM J. Comput.* 31(2), pp. 496–526, 2001.

[8] A. A. Razborov and S. Rudich, "Natural proofs," *J. Comput. System Sci.* 55(1), pp. 24–35, 1997.

[9] S. Neph, "satfactor," GitHub. `https://github.com/sjneph/satfactor`.

[10] M. Schaller and R. Schützhold, "Factoring semi-primes with (quantum) SAT-solvers," *Scientific Reports* 12, 2022.

[11] P. W. Shor, "Algorithms for quantum computation," *Proc. 35th IEEE FOCS*, pp. 124–134, 1994.

[12] V. Shoup, "Lower bounds for discrete logarithms and related problems," *Proc. EUROCRYPT*, LNCS 1233, pp. 256–266, 1997.

[13] D. Yurichev, "Encoding Basic Arithmetic Operations for SAT-Solvers." `https://yurichev.com/mirrors/SAT_factor/`.